

DATABASE AUTHORIZATION AND PRIVILEGES

Learning Objectives

By the end of this lecture, students should be able to:

- Differentiate the types of data authorizations and privileges in databases.
- Explain how SQL enforces authorization checks.
- Recognize the role of the database administrator (DBA).
- Explain how privileges are granted, revoked, and transferred in SQL.
- Understand the concept of roles and role-based authorization.
- Describe how authorizations apply to views.
- Explain schema-level authorizations and the references privilege.
- Describe cascading revocation and the grant option.
- Recognize the concept of row-level authorization and its implementation.

Introduction to Authorization

- **Authorization:** The process of granting users' specific rights to access or manipulate parts of a database.
- Each right is called a **privilege**.

Types of Data Authorizations

Users may be granted privileges on relations or views, such as:

- **Read (SELECT)** → Permission to view data.
- **Insert (INSERT)** → Permission to add new data.
- **Update (UPDATE)** → Permission to modify existing data.
- **Delete (DELETE)** → Permission to remove data.

Schema-Level Authorizations

Beyond data, users may also be authorized to manage the **database schema**:

- Create new relations.
- Modify existing relations.
- Drop relations.

Enforcement of Authorization

- When a query or update is submitted, the SQL system checks if the user has the required privileges.
- Unauthorized operations are **rejected** immediately.

Granting and Revoking Privileges

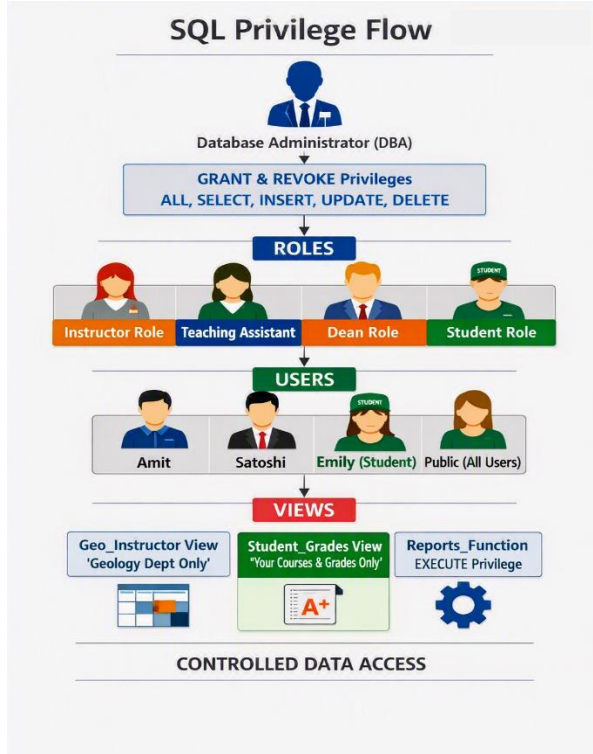
- Authorized users may **grant** privileges to others.
- They may also **revoke** privileges previously granted.
- This allows flexible delegation of access rights.

Role of the Database Administrator (DBA)

- The DBA holds the **ultimate authority** in the system.

- Responsibilities include:
 - Authorizing new users.
 - Restructuring the database.
 - Managing overall security and access.
- Comparable to a **superuser** or **system administrator** in operating systems.

Granting, Revoking, and Managing Privileges in SQL



Granting Privileges

- SQL supports privileges: SELECT, INSERT, UPDATE, DELETE.
- ALL PRIVILEGES is shorthand for granting all available rights.
- A user who creates a relation automatically receives all privileges on it.
- GRANT statement syntax:
*GRANT <privilege list>
ON <relation/view>
TO <user/role list>;*
- Example:
GRANT SELECT ON department TO Amit, Satoshi;
→ Grants SELECT privilege on the *department* relation.

Revoking Privileges

- Privileges can be withdrawn using the **REVOKE statement**:
- Syntax:
*REVOKE <privilege list>
ON <relation/view>
FROM <user/role list>;*

- Example:
REVOKE UPDATE (budget) ON department FROM Amit, Satoshi;

Roles in SQL

- **Roles** simplify privilege management by grouping authorizations.
- Example roles: *Instructor, Teaching Assistant, Student, Dean*.
- Roles are created and managed with SQL commands:
- Syntax:
CREATE ROLE instructor;
GRANT SELECT ON takes TO instructor;
GRANT instructor TO dean;
GRANT dean TO Satoshi;
- Privileges of a user = direct privileges + privileges of roles assigned (including inherited chains).
- Benefits:
 - Easier management of large groups.
 - Users connect with their own IDs, ensuring accountability.

Authorization on Views

- Views allow **restricted access** to subsets of data.
- Example:
CREATE VIEW geo_instructor AS
*(SELECT * FROM instructor WHERE dept_name = 'Geology');*
- A staff member can query only the *geo_instructor* view, not the full *instructor* relation.
- A view creator only gets privileges consistent with their existing rights.
- Update privileges on a view require update privileges on underlying relations.
- SQL checks authorization before expanding a query into base relations.

Functions and Procedures

- Privileges can also apply to **functions and procedures**.
- **EXECUTE privilege** allows running them.
- SQL:2003 introduced **SQL SECURITY INVOKER**, meaning functions run under the invoker's privileges instead of the definer's.
- This supports shared libraries with consistent access control.

```
CREATE DATABASE school;  
USE school;
```

```
--Create a simple procedure:  
DELIMITER //
```

```
CREATE PROCEDURE GetStudentCount()  
BEGIN
```

```
SELECT COUNT(*) AS TotalStudents
FROM students;
END //

DELIMITER ;

GRANT EXECUTE
ON PROCEDURE school.GetStudentCount
TO 'amit'@'localhost';

or

GRANT EXECUTE
ON PROCEDURE school.GetStudentCount
TO 'instructor';
```

Authorizations on Schema

- Only the schema owner can modify the schema (create/delete relations, add/drop attributes or indices).
- SQL introduces the REFERENCES privilege to control foreign key creation.
- Example:

GRANT REFERENCES (dept_name) ON department TO Mariano;

→ Allows Mariano to create relations referencing dept_name as a foreign key.

Suppose the department table contains:

```
CREATE TABLE department (
    dept_name VARCHAR(50) PRIMARY KEY,
    building VARCHAR(50),
    budget DECIMAL(12,2)
);
```

Mariano wants to create another table, instructor, where dept_name refers to the department.dept_name column:

```
CREATE TABLE instructor (
    instructor_id INT PRIMARY KEY,
    name VARCHAR(100),
    dept_name VARCHAR(50),
    FOREIGN KEY (dept_name)
        REFERENCES department(dept_name)
);
```

- Why restrict this?
 - Foreign keys affect other users' ability to delete or update referenced data.

- Prevents unintended constraints on shared relations.
- The **REFERENCES privilege** is also needed for **check constraints** that reference other relations.

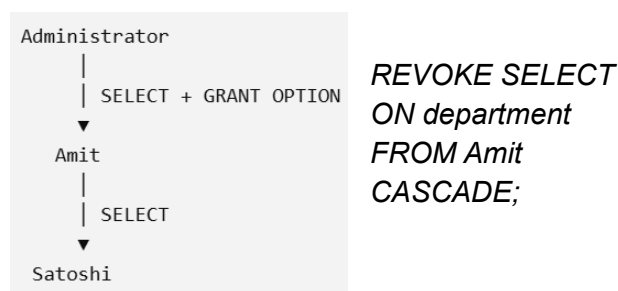
Transfer of Privileges

- Privileges can be **passed on** using the WITH GRANT OPTION clause.
- WITH GRANT OPTION can pass the SELECT privilege to other users
- Example:
GRANT SELECT ON department TO Amit WITH GRANT OPTION;
- The creator of an object automatically holds all privileges, including the ability to grant them.
- Privilege delegation can be visualized using an authorization graph:
 - Nodes = users
 - Edges = privilege grants
 - Root = database administrator (DBA)
 - A user has authorization if there's a path from the DBA to that user.

Revoking of Privileges

- Revocation may **cascade** — removing privileges from one user can affect others who received privileges indirectly.
- Example:
REVOKE SELECT ON department FROM Amit, Satoshi RESTRICT;
- RESTRICT prevents cascading revocation (system raises an error if dependencies exist).
- You can revoke only the grant option:
REVOKE GRANT OPTION FOR SELECT ON department FROM Amit;
- CASCADE allows automatic revocation down the chain (default behavior).

The privilege chain is:



- SQL also supports role-based granting using GRANTED BY CURRENT ROLE, ensuring privileges persist even if the granting user loses their role.

Row-Level Authorization

- Provides fine-grained control at the tuple level.
- Example scenario: students can view only their own records in the takes relation.

- For **row-level authorization**, the goal is to allow each student to view **only their own records** in the takes table, not the records of other students.
- If your takes table has a student_id column, a common approach is to create a view that filters rows based on the currently logged-in user.

```
CREATE VIEW my_takes AS SELECT student_id, course_id, semester, year,  
grade FROM takes WHERE student_id = CURRENT_USER();
```

```
GRANT SELECT ON my_takes TO 'student'@'localhost';
```

Key Takeaways

- Authorization ensures controlled access to data and schema.
- Privileges define what actions a user can perform.
- SQL enforces strict checks before executing queries.
- The DBA is the central authority, ensuring security and proper delegation.
- **GRANT** and **REVOKE** manage user privileges.
- **Roles** streamline authorization for groups of users.
- **Views** provide controlled, partial access to data.
- SQL enforces strict checks to maintain security and accountability.
- Schema modifications are restricted to schema owners.
- The **REFERENCES privilege** prevents unwanted foreign key dependencies.
- Privileges can be delegated using **WITH GRANT OPTION**.
- **Cascading revocation** ensures consistent privilege removal.
- **Row-level authorization** enables personalized data access for users.

Reference:

- Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Database System Concepts* (7th ed.). McGraw-Hill.